LAB 5: STATIC CODE ANALYSIS — REFLECTION REPORT

Department of Computer Science and Engineering | Software Engineering Lab

Student Name: Ashwin Harish

SRN: PES1UG24AM900

Course: Software Engineering (SE) Lab

Deliverable: 5. Reflection Question and Answers

Repository: https://github.com/ash29032006/L5_62_scramble

Date: October 23, 2026

1. Differences in Purpose, Scope, and Rulesets (Pylint, Flake8, and Bandit)

Static code analysis tools inspect source code without executing it, but each tool operates at a distinct abstraction level with different analytical goals:

- **Flake8 (Formatting and Style Linter):** Flake8 combines Pyflakes, pycodestyle, and McCabe. It focuses strictly on surface-level PEP 8 formatting conventions (whitespace, blank lines, line lengths under 79 characters) and basic syntax issues like unused variables. It parses code quickly without full Abstract Syntax Tree (AST) inference, making it suitable for rapid pre-commit hooks.
- **Pylint (Comprehensive Quality and Architecture Analyzer):** Pylint builds a deep AST using astroid to analyze structural and semantic design. It evaluates code maintainability, docstrings, variable scopes, naming conventions, and structural complexity (such as excessive branching, large parameter lists, or oversized functions). Its 10-point scoring model gives teams an objective metric to track codebase health over time.
- **Bandit (Security Vulnerability Scanner):** Bandit queries AST representations against known security vulnerability patterns from Common Weakness Enumeration (CWE) databases. It flags insecure cryptographic primitives, unseeded or deterministic pseudo-random generators (B311), hardcoded secrets (B105), unsafe shell command execution, and dangerous deserialization.

By combining all three tools, developers establish a defense-in-depth pipeline covering style consistency (Flake8), structural reliability and maintainability (Pylint), and security vulnerability prevention (Bandit).

2. Critical Vulnerabilities and Logical Bugs Uncovered

Static analysis uncovered several key vulnerabilities and architectural flaws in the Word Scramble codebase:

- **Deterministic Randomness (Bandit B311):** The game originally used Python's random.choice and random.shuffle, which rely on the Mersenne Twister PRNG. Because Mersenne Twister is deterministic, an observer can reconstruct its internal 624-word state and predict upcoming secret words and shuffle orders. Migrating to Python's secrets module solved this by deriving randomness from the operating system's cryptographic entropy source (/dev/urandom).
- **Plaintext Secret Heuristic (Bandit B105):** Declaring 'secret_word = "" triggered Bandit's credential detection rule. Renaming the internal attribute to target_word and exposing a property getter and setter eliminated the false positive while safeguarding future maintainability.
- **Monolithic Rendering Scope (Pylint R0914, R0915):** The original render() method contained 34 local variables and 68 statements, handling rack calculations, timer bar geometry, button rendering, and feedback display in a single block. This high coupling created maintainability hazards where changing one UI element risked breaking others. Decomposing it into five isolated helper functions resolved scope pollution.
- **Excessive Cyclomatic Complexity (Pylint R0912):** handle_event originally contained 18 branches, mixing keyboard events, mouse clicks, drag initiation, and drop target checks. Modularizing this into helper methods separated concerns cleanly.

3. Practical Challenges, Trade-offs, and Remediation Strategies

Several practical engineering trade-offs were balanced during remediation:

• **Preventing Functional Regressions During Refactoring:** Breaking large methods like render() and handle_event() into helper methods required careful preservation of drag-and-drop coordinates, mouse offsets, and visual tile layouts. Gameplay was verified after each commit to ensure identical user experience.

• **Handling Pygame Dynamic Member Injections (Pylint E1101):** Pygame binds attributes (such as pygame.QUIT, pygame.Rect, and key constants) dynamically through C-extensions, triggering false-positive 'no-member' warnings in Pylint. Rather than globally disabling checks, targeted in-file pragmas were used to suppress false positives while maintaining strict quality enforcement on application logic.

• **PEP 8 79-Character Boundary vs Readability (Flake8 E501):** Strict line-length limits can produce fragmented code when formatting graphical coordinates and RGB color tuples. Statements were formatted with multiline parenthesized grouping, maintaining both PEP 8 compliance and visual readability.

4. Role and Value in the Software Development Life Cycle (SDLC)

Integrating static analysis into the SDLC provides measurable engineering advantages:

• **Shift-Left Defect Detection:** Discovering style violations, code smells, and security oversights during local development is dramatically cheaper than discovering bugs during integration testing or production outages.

• **Automated CI/CD Quality Gates:** Linters can be embedded directly into automated pull request workflows (e.g., GitHub Actions). Merges can be blocked if Pylint score falls below 9.5 or if Bandit flags vulnerabilities.

• **Code Consistency and Reduced Review Overhead:** Automated formatting enforcement (Flake8) prevents code style debates in code reviews, allowing peer reviewers to focus on architectural design and business logic.

• **Technical Debt Reduction:** Requiring docstrings, minimal nesting, and low cyclomatic complexity keeps codebases accessible and clean as team size and code volume grow.